With this version of our test, we no longer need to deal with MetricsReporter instances. Instead, we have a simpler interface for incrementing metrics. We could potentially clean up metric-counting code all over our system. We are, however, back to using partial doubles again, something we generally avoid.

Stubbing a Constant

As we've seen in this chapter, having both real and fake behavior in the same object can cause problems. We'd prefer to use a purely fake reporter. We can achieve this goal by creating a class_double and stubbing the MetricsReporter constant to return that double:

15-using-test-doubles-effectively/13/api_request_tracker/spec/api_request_tracker_spec.rb

```
it 'increments the request counter' do
  reporter = class_double(MetricsReporter)
  stub_const('MetricsReporter', reporter)
  expect(reporter).to receive(:increment).with('api.requests.get_users')

  APIRequestTracker.new.process(request)
end
```

This pattern is useful enough that RSpec provides a way to implement it in one line of code. Just tack as_stubbed_const onto the end of your class double, and it will automatically replace the original class—just for the duration of the example:

15-using-test-doubles-effectively/14/api_request_tracker/spec/api_request_tracker_spec.rb

```
it 'increments the request counter' do
  reporter = class_double(MetricsReporter).as_stubbed_const
  expect(reporter).to receive(:increment).with('api.requests.get_users')

  APIRequestTracker.new.process(request)
end
```

We like the way that stubbed constants allow us to use a pure double. The downside is that they add their fake behavior implicitly. Someone reading the APIRequestTracker code is not going to suspect that the MetricsReporter constant might refer to a test double.

Stubbed constants can surface dependencies hiding in our code. When we hardcode the name of a class, we are tightly coupling our code to that specific class. Sandi Metz discusses ways to deal with this antipattern in *Practical Object-Oriented Design in Ruby [Met12]*.

To prevent this tight coupling, we prefer to depend on abstract roles rather than concrete classes. We'd like this API request tracker to work with *any* object that can report metrics (or pretend to report metrics), not just MetricsReporter instances.